Solution: Minimum jumps with fee

Problem Statement

Given a staircase with ‘n’ steps and an array of 'n' numbers representing the fee that you have to pay if you take the step. Implement a method to calculate the minimum fee required to reach the top of the staircase (beyond the top-most step). At every step, you have an option to take either 1 step, 2 steps, or 3 steps. You should assume that you are standing at the first step.

Example 1:

```
Number of stairs (n) : 6
Fee: {1,2,5,2,1,2}
Output: 3
Explanation: Starting from index '0', we can reach the top through: 0->3->top
The total fee we have to pay will be (1+2).
```

Example 2:

```
Number of stairs (n): 4
Fee: {2,3,4,5}
Output: 5
Explanation: Starting from index '0', we can reach the top through: 0->1->top
The total fee we have to pay will be (2+3).
```

Let's first start with a recursive brute-force solution.

Basic Solution

At every step, we have three options: either jump 1 step, 2 steps, or 3 steps. So our algorithm will look like:

Java

```
class Solution {

  public int findMinFee(int[] fee) {
    return findMinFeeRecursive(fee, 0);
  }

  private int findMinFeeRecursive(int[] fee, int currentIndex) {
    if( currentIndex > fee.length - 1)
      return 0;
```

```
    // if we take 1 step, we are left with 'n-1' steps;
    int take1Step = findMinFeeRecursive(fee, currentIndex + 1);
    // similarly, if we took 2 steps, we are left with 'n-2' steps;
    int take2Step = findMinFeeRecursive(fee, currentIndex + 2);
    // if we took 3 steps, we are left with 'n-3' steps;
    int take3Step = findMinFeeRecursive(fee, currentIndex + 3);

    int min = Math.min(Math.min(take1Step, take2Step), take3Step);

    return min + fee[currentIndex];
  }

  public static void main(String[] args) {
```

Run Code Result | Stdout

You must run your code first

Execute

The time complexity of the above algorithm is exponential $O(3^n)$. The space complexity is $O(n)$ which is used to store the recursion stack.

Top-down Dynamic Programming with Memoization

To resolve overlapping subproblems, we can use an array to store the already solved subproblems. Here is the code:

Java

```
class Solution {

  public int findMinFee(int[] fee) {
    int dp[] = new int[fee.length];
    return findMinFeeRecursive(dp, fee, 0);
  }

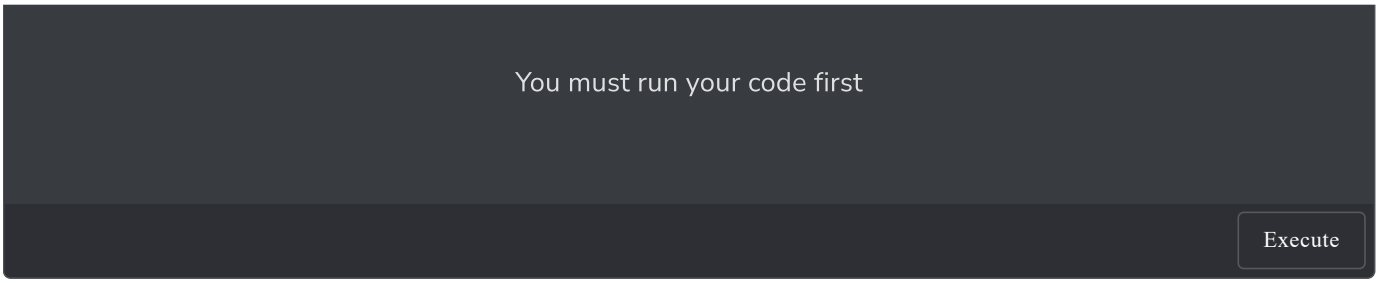
  private int findMinFeeRecursive(int[] dp, int[] fee, int currentIndex) {
    if( currentIndex > fee.length - 1)
      return 0;

    if(dp[currentIndex] == 0) {
      // if we take 1 step, we are left with 'n-1' steps;
      int take1Step = findMinFeeRecursive(dp, fee, currentIndex + 1);
      // similarly, if we took 2 steps, we are left with 'n-2' steps;
      int take2Step = findMinFeeRecursive(dp, fee, currentIndex + 2);
      // if we took 3 steps, we are left with 'n-3' steps;
      int take3Step = findMinFeeRecursive(dp, fee, currentIndex + 3);

      dp[currentIndex] = fee[currentIndex] + Math.min(Math.min(take1Step, take2Step), take3Step);
    }

    return dp[currentIndex];
  }
```

Run Code Result | Stdout

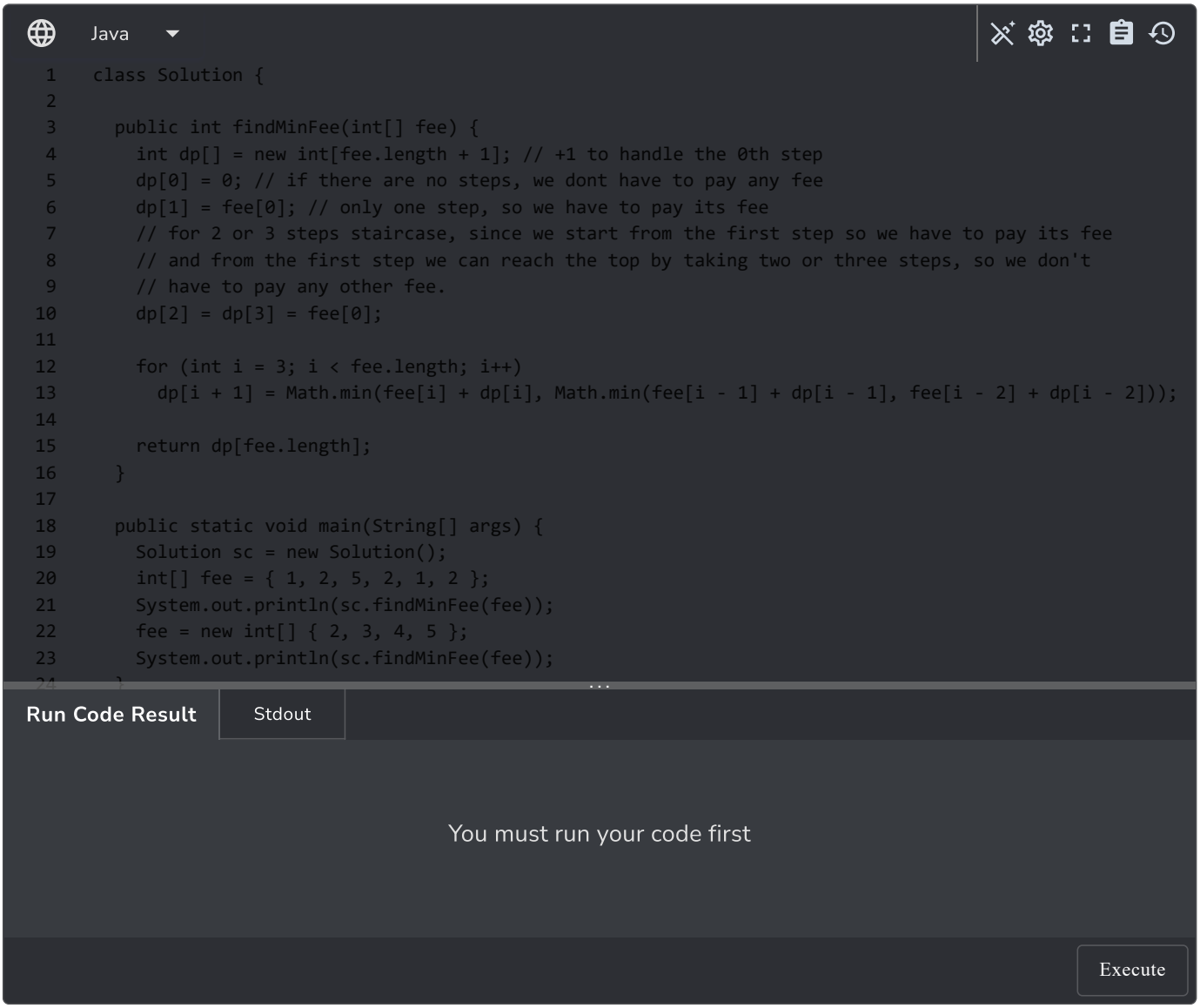


Bottom-up Dynamic Programming

Let's try to populate our `dp[]` array from the above solution, working in a bottom-up fashion. As we saw in the above code, every `findMinFeeRecursive(n)` is the minimum of the three recursive calls; we can use this fact to populate our array.

Code

Here is the code for our bottom-up dynamic programming approach:



The above solution has time and space complexity of $O(n)$.

Fibonacci number pattern

We can clearly see that this problem follows the Fibonacci number pattern. The only difference is that every Fibonacci number is a sum of the two preceding numbers, whereas in this problem every number (total fee) is the minimum of previous three numbers.

← Previous

Minimum Jumps With Fee

Next →

House Thief

Mark as Completed